



CIS 1902: Python

Week 0

Ash Fujiyama



Scheduled Topics

- Introduction
 - Course Logistics
 - Python Introduction + Activity
 - (If time) Environment setup
- 

The background is a dark teal color. There are several vertical lines of small, light teal squares. On the left side, there are two lines of squares, one with 12 squares and one with 8 squares. On the right side, there are three lines of squares, with 8, 12, and 10 squares respectively. The word "Introduction" is centered in the middle of the slide in a large, white, sans-serif font.

Introduction

Welcome!

- My name is Ash Fujiyama (they/them)
- Currently pursuing a masters in CIS, graduated undergrad here too!
- I've been a TA for 5 semesters, head TA for 2
- **I'm really excited to be teaching!**

Contact:

- afuji@seas.upenn.edu
- Office Hours: Levine 269C, time TBD!
(I will be asking y'all for what times best work)
- You are always welcome to email me to schedule a 1-on-1 meeting



A bit more about me:

- I have a wonderful cat named Fresno
- I play rhythm games (both mobile and arcade)
- My favorite classes at Penn:
 - Compilers, Operating Systems, Software Systems
 - ASL I-III
- Other hobbies: Magic the Gathering, rock climbing



A bit more about your TA:

- Hi! My name is Foram Shah
- Currently a third year student pursuing a BSE in CS
- Love reading or going for a swim in my free time

Contact:

- shahf@engineering.upenn.edu
- Office Hours: TBD
- Feel free to reach out to chat about the course, questions you have, or just in general!



A bit more about you:

- Name?
- Year?
- One thing you enjoyed from break

About The Course

Why Python?

- Simple, readable syntax

Why Python?

- Simple, readable syntax
- *Very* popular language
 - 4th most used language in the Stack Overflow survey for all respondents and professional developers
 - Most popular language for those learning to code

Why Python?

- Simple, readable syntax
- Very popular language
 - 4th most used language in the Stack Overflow survey for all respondents and professional developers
 - Most popular language for those learning to code
- Powerful language, many uses
 - Data Science & ML
 - Web Development: Google, YouTube, Reddit
 - Quickly throwing together scripts for random tasks
 - Many libraries for many tasks: web scraping, image processing, games, NLP, etc.

What You Need Going in:

- Some amount of programming knowledge in another language
 - Foundational topics: variables, loops, conditionals
 - Comfortability in writing and debugging small programs (10-100 lines)
- Willingness to make mistakes and learn from them / ask for help

What's In This Course

Foundational Pythonic programming skill development

- Syntax
- Data types, data structures, iterators, generators
- Classes & modules (how to organize code)
- Scripting & testing
- Package management: pip, venv
- Popular packages in data science/visualization and ML:
 - Numpy, pandas
 - Matplotlib
 - Scikit-learn, pytorch

Logistics

- Lectures: once a week
 - Activities to practice in class
 - I will post slides before class so you can reference them during activities
- Homeworks: due 2 weeks after assignment*
 - Assigned every other week
 - Resubmission Policy (more on this next slide)
 - Infinite submissions to Gradescope's autograder
 - Style grading done manually
- Worksheets: due 1 week after assignment
 - Completion based grading
 - We may provide feedback
- Assignment due dates: Mondays @ 11:59 pm

Resubmission Policy

- Resubmissions can be requested on odd numbered weeks.
- Google form will be found on Ed and on the course website
- 1 hw opened per resub request
- Due 2 weeks after assignment reopens

Resubmission Policy

- Resubmissions can be requested on odd numbered weeks.
- Google form will be found on Ed and on the course website
- 1 hw opened per resub request
- Due 2 weeks after assignment reopens

Why this and not a late-policy?

- More flexibility on when you want to work on old homework assignments
- Mastery grading: a chance to show you've learned/improved from when you first were introduced to the material
 - Actually getting to use our style-grading feedback!

Grading

- ~6 Homework: 70%
- Worksheets: 20%
- Participation: 10%

Participation is graded based on class attendance and Ed participation. We highly encourage asking questions and engaging with others both in class and online on Ed. We hope that with a small class size, you all will have the chance to get to know both instructors and other students better than in a traditional CIS lecture course.

Resources

- Course website will contain almost all information, including links to other sites
 - Gradescope
 - Ed
 - Python Documentation (including the PEP 8 style guide)
- Please reach out if you are not sure where something is located

Python Start!

Python versus Java

```
# prints odd integers less than x
def loop(x):
    """
    This is a multi-line comment.
    Can be used for docstrings
    """
    for i in range(x):
        if i % 2 != 0:
            print(i)

def main():
    x = 11
    loop(x)

if __name__ == "__main__":
    main()
```

```
class Main {
    // prints odd integers less than x
    /**
     * This is a multi-line comment.
     * Can be used as docstrings
     */
    public static void loop(int x) {
        for (int i = 0; i < x; i++) {
            if (i % 2 != 0) {
                System.out.println(i);
            }
        }
    }

    public static void main(String[] args) {
        int x = 11;
        loop(x);
    }
}
```

Python versus Java

- `#` for inline comments, `"""` (triple quotes) for multi-line comments
- `def` keyword for functions
- No declaring variable type: for local variables, function arguments, or return type!
- Simpler print function name: `print()`
- Use indents and colons to organize code blocks and define scope
- No `()` needed for conditionals or loop guards
- `if __name__ == '__main__':` is used as the entry point to the program
 - **Must be at the bottom of your file!**

Working with Python

- Python is an **interpreted** language
 - You may be familiar with needing to **compile** a program before being able to run it (Java, C, C++)
 - Source file can be run directly
 - Consequence: faster debug cycles, slower runtime
- .py files, or interactive environment (REPL: Read Evaluate Print Loop)

Variables

- Don't need type declaration when initializing new variables, Python figures it out for you
- Dynamically typed
- Python won't let you declare variables without initialization

```
x = 39
print(type(x)) # prints: <class 'int'>

x = "Miku" # this is allowed
print(type(x)) # prints: <class 'str'>

y # throws an error!
```

What If We Want To Declare Still?

- Python has its own null, called `None`
- Can declare a variable without setting any meaningful initial value by initializing with `None`
- When you are able to give the intended value to the variable, can reassign it without issue
 - Dynamic typing!!

```
x = 39
y = None # no more error!
print(type(y)) # prints: <class 'NoneType'>

y = x + 10 # y is now "properly" initialized
print(type(y)) # prints: <class 'int'>
```


Basic Types

- Integer (int)
- String (str)
- Boolean (bool)
- Float (float)

Basic Types

Type conversion/casting: convert from one type to another

- **Implicit:** when Python does it automatically (usually to prevent data loss)
- **Explicit:** when you are the one directing the interpreter to change a value's type

```
x = 5  
y = float(x)  
z = x + 0.0
```

What are the types of `x`, `y`, and `z`?

What are x and y's type? Is this Legal?

```
x = 5  
y = float(x)
```

```
x = 5.3  
y = bool(x)
```

```
x = 'abc'  
y = int(x)
```

```
x = '5'  
y = int(x)
```

```
x = '5'  
y = int(x)
```

Basic Types

- Integer (int)
- String (str)
- Boolean (bool)
- Float (float)

Beware! Integer division gives a float! (why is this?)

```
x = 3.9
print(x) # prints: 3.9
print(type(x)) # prints: <class 'float'>
x = int(x) # converts x to an int
print(x) # prints: 3
print(type(x)) # prints: <class 'int'>
y = x / 2 # gets a float after division starting with python version 3
print(y) # prints: 1.5!
```

Basic Types

- Integer (int)
- String (str)
- Boolean (bool)
- Float (float)

Be wary of bool behavior!

```
x = True
```

```
print(x) # prints: True
```

```
print(x + 1) # prints 2
```

```
x = not True
```

```
print(x) # prints: False
```

```
print(x + 1) # prints: 1
```

Comments and Docstrings

scratchwork >  scrap.py > ...

```
1  import math
2  def cartesian_to_polar(x, y):
3      """
4      converts cartesian coordinates (x, y) into polar coordinates (r, theta)
5
6      :param x: x coordinate (float)
7
8      :param y: y coordinate (float)
9      """
10     return (math.sqrt(x*x + y*y), math.atan(y/x))
11
12 def main():
13     r, theta = cartesian_to_polar(1, 2)
14
15
16 if __name__ == "__main__":
17     main()
```

Comments and Docstrings

scratchwork >  scrap.nv >

```
1  import math
2  def cartesian_to_polar(x: Any, y: Any) -> tuple[float, float]:
3      """
4      converts cartesian coordinates (x, y) into polar coordinates (r, theta)
5
6      :param x: x coordinate (float)
7
8      :param y: y coordinate (float)
9      """
10     return (math.sqrt(x**2 + y**2), math.atan2(y, x))
11
12 def main():
13     r, theta = cartesian_to_polar(1, 2)
14
15
16 if __name__ == "__main__":
17     main()
```

(function) def cartesian_to_polar(
x: Any,
y: Any
) -> tuple[float, float]

x: Any,

y: Any

) -> tuple[float, float]

eta)

converts cartesian coordinates (x, y) into polar coordinates (r, theta)

Parameters

x

x coordinate (float)

y

y coordinate (float)

Control Flow

- By default, code will run from top to bottom, line by line
- But we have ways to change that:
 - **Branching:** creating two or more options for the next line to run
 - **Looping:** executing the same code multiple times
 - **Jumping:** choosing a different line to run next
- You may know these as:
 - If/else if/else branches
 - for/while loops
 - Function calls

Conditionals

- `if`, `elif` (else if), and `else`
- Conditions guarding the if/elif statements must evaluate to `True` or `False`

Conditionals

```
afujiyama@Rikyuu-on-PC scratchwork % python3
```

```
>>> x = 30 + 34
```

```
>>> x == 64
```

```
True
```

```
>>> if x == 64:
```

```
...     print("this will print")
```

```
...     print("this is in the same code block and will also print")
```

```
...
```

```
this will print
```

```
this is in the same code block and will also print
```

```
>>> if x == 42:
```

```
...     print("64 == 42?? idts.")
```

```
... elif x == 64:
```

```
...     print("64 == 30 + 34")
```

```
... else:
```

```
...     print("64 != 42 and 64 != 30 + 34")
```

```
...
```

```
64 == 30 + 34
```

Iteration: while loop

```
>>> count = 0
```

```
>>> while count < 5:
```

```
...     print("count is less than 5")
```

```
...     count += 1
```

```
...     break ← break means exit the loop
```

```
...
```

```
count is less than 5
```

```
>>> print(count)
```

```
1
```

Iteration: while loop

```
>>> count = 0
```

```
>>> while count < 5:
```

```
...     count += 1
```

```
...     continue
```

```
...     print("count is less than 5")
```

```
...
```

```
>>> print(count)
```

```
5
```

```
>>>
```

← continue means skip the rest of the current iteration and start the next one

Iteration: for loop

- Typical Java for-loop syntax: `for (int i = 0; i < n; i++) {}`
 - Defined by start + condition + step
- Python for-loop syntax is meant to iterate through any sequence: `for ... in ... :`
 - Means: for each item (name) in sequence
 - Doing something with each item (the body of the loop)

Example:

```
>>> for i in range(5):  
...     print(i)
```

← `range()` generates a sequence of integers starting with 0 and stopping before the passed argument (5)

...

0

1

2

3

4

Iteration Over Other Sequences

```
>>> x = "pineapple buns"
>>> for i in x:
...     print(i)
...
p
i
n
e
a
p
p
l
e

b
u
n
s
```

Sequences

- Sequence: an arrangement of things that come one after another
- We just saw two examples:
 - `range(5)` yields the sequence: 0 1 2 3 4
 - Strings are a sequence of characters
- Properties:
 - **Length:** how many items are in the sequence
 - `len(range(5)) = 5`
 - `len("hello") = 5`
 - **Membership:** is a particular item located in the sequence?
 - `'I' in 'team'` will give us False
 - **Index:** where is a particular item located in the sequence
 - `range(5)[3]` will give us 3
 - If `x == "hello"`, then `x[4]` returns the character at index 4. Which character is that?

Quick Note: Indexing

- Many things in Python starts with 0 and not 1!
 - Range values start at 0 by default, e.g. `range(5)`: 0, 1, 2, 3, 4
 - Indexing starts at 0, `x[1]` will give you the second element of `x`!
- Consequence: given a sequence `x`, `x[len(x)]` will return an error for being out of bounds
- Most of the popular languages are zero-indexed
 - Some well known 1-indexed languages: Fortran, COBOL, MATLAB, R
- Python also has negative indexing: -1 through `-len(x)`
 - `range(5)[-1] = 4`
 - `range(5)[-5] = 0`

Ed Lessons



Environment Setup

CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)